PayFlow Africa — Target AWS Architecture Design

Document: Architecture Design & Defense
Project: Cloud Modernization & Migration
Client: PayFlow Africa
Date: 2026
Prepared by: Lead Cloud Solutions Architect

Executive Summary

PayFlow Africa requires a cloud architecture that achieves **99.99% availability** while processing up to **2,500 TPS** during extreme events, supporting **24/7 payment operations**, maintaining **strong financial transaction integrity**, and remaining cost-effective at **~₦200M/year** operating cost.

The proposed architecture uses a **Strangler Pattern** to gradually modernize the monolith, **ECS Fargate** for compute resilience and team simplicity, **RDS Multi-AZ PostgreSQL** as the financial system of record, and **Multi-AZ deployment** for automatic failure isolation and recovery.

This design prioritizes **reliability over cost**, **operational simplicity over complexity**, and **incremental risk reduction over big-bang transformation**.

---

1. Architecture Pattern: Strangler Approach

Pattern Choice: Monolith → Modular Monolith → Gradual Service Extraction

What this means:

```
Phase 1 (Months 1-3):  Keep monolith operational in cloud
                       Extract payments service (most critical)
                       Establish deployment safety & observability

Phase 2 (Months 4-6):  Monolith + Payments Service running
```

```
                        Customer/auth remains in monolith
                        Reporting/analytics extracted as needed

Phase 3 (Year 2):       Continue extracting: User Management → separate service
                        Merchant Management → separate service
                        Notifications → separate service
                        Monolith shrinks but doesn't disappear overnight
```

Why This Pattern?

Why NOT full microservices immediately:

- Would require complete rewrite (violates 6-month timeline)
- 18 engineers can’t build and operate 10 microservices in 6 months
- Financial transaction complexity demands deep testing
- Introduces distributed systems problems (eventual consistency, network latency) too early

Why NOT keep monolith as-is:

- Cannot deploy safely today (entire system at risk)
- Doesn’t scale payment operations independently
- Blocks progress on reliability and delivery velocity

Why Strangler works:

- Keeps PayFlow operational during migration
- Allows team to learn AWS incrementally
- De-risks payment processing first (highest value, highest risk)
- Provides clear boundaries for gradual extraction
- Enables parallel work (payments team extracts payment service while others maintain monolith)
- Can adapt based on what we learn in months 1-3

Architectural Implication

The payments service becomes the **first extracted domain**. This isolates the most financially sensitive component, allows independent scaling, enables safer deployments, and proves the “extraction capability” before doing it again with other

services.

---

2. High-Level Request Flow

Customer Payment: From Initiation to Confirmation

```
┌──────────────────────────────────────────────────────────────┐
│                    CUSTOMER PAYMENT FLOW                     │
└──────────────────────────────────────────────────────────────┘

1. ENTRY POINT
   Mobile App / Web → TLS 1.3 → CloudFront (caching) → ALB

2. API GATEWAY LAYER
   ALB → API Gateway (rate limiting, auth validation, request logging)

   Action: Validate JWT token (existing auth system retained)
   Idempotency Key: Required header (prevents duplicate processing)

3. SYNCHRONOUS PATH (Core payment processing)
   API → Payments Service (ECS/Fargate)

   Steps:
   a) Validate request (amount, merchant, customer)
   b) Reserve transaction in database (atomic write)
   c) Call external payment processor (Flutterwave, bank, card network)
      - Timeout: 8 seconds (don't hang entire request)
      - Retry logic: Provider-aware (some idempotent, some not)
      - Circuit breaker: If provider fails 10x → fail fast
   d) Commit transaction to RDS (if provider approved)
   e) Return confirmation to customer (< 2 seconds typical)

4. ASYNCHRONOUS PATH (Post-confirmation processing)
   Confirmation sent to customer → Message queued

   Steps:
   a) Write event to SQS (transaction confirmed, ledger updated, etc.)
   b) Lambda/worker processes: Notifications, analytics, reconciliation
   c) No timing dependency (customer already confirmed)

5. EXTERNAL DEPENDENCIES
   Payment Processor (Flutterwave, Paystack, banks)
   ├─ Timeout: 8s (configured per provider)
   ├─ Retry: Only for idempotent operations
```

```
   └─ Fallback: If provider down > 60s, queue for retry

   Ledger Database (RDS Multi-AZ PostgreSQL)
   ├─ Transaction record (system of record)
   ├─ Idempotency table (prevents duplicate charges)
   └─ Synchronous writes only (no eventual consistency)

6. DATA CONSISTENCY GUARANTEE
   If customer doesn't get confirmation:
   ├─ Money NOT charged (provider call failed before commit)

   If customer gets confirmation:
   ├─ Money IS in ledger (atomic write succeeded)
   ├─ Impossible to charge twice (idempotency key checked)

7. CONFIRMATION TO CUSTOMER
   < 2 seconds for successful transactions
   < 5 seconds for provider timeouts (queued for async retry)
```

Key Design Decisions Embedded in This Flow

1. **Synchronous core path** — Financial transactions must wait for payment processor response (no fire-and-forget)
2. **Timeout boundaries** — 8 seconds per provider call (prevents cascading hangs)
3. **Idempotency keys** — Every request has unique key (stored in database, checked before processing)
4. **Asynchronous notifications** — Send confirmation immediately, email/SMS/webhook processed later
5. **External provider abstraction** — Retry/timeout/circuit-breaker logic per provider (not one-size-fits-all)

---

3. Compute Architecture: ECS Fargate on Multi-AZ

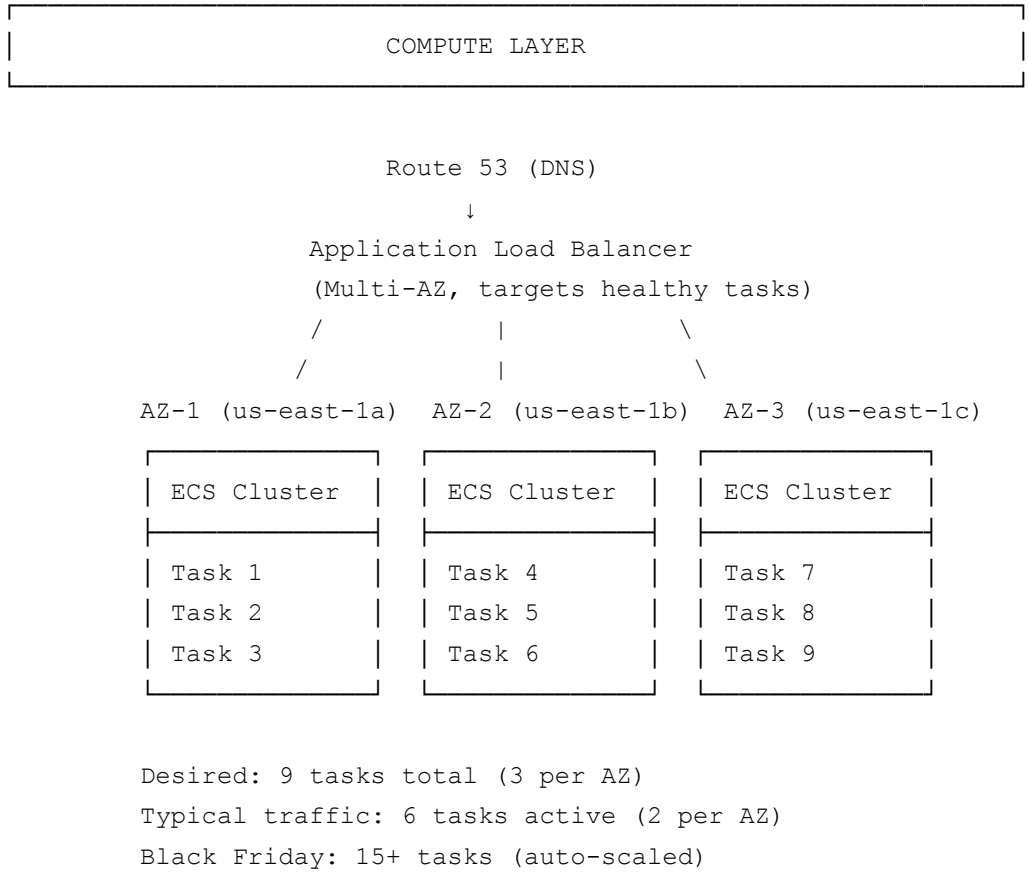
Service: AWS ECS (Elastic Container Service) with Fargate Launch Type

What this means:

- Application runs in Docker containers
- Containers managed by ECS (not manually)

- Infrastructure managed by AWS Fargate (not by us)
- Deployed across multiple Availability Zones

Topology



Why ECS Fargate (Not EC2, Not EKS)?

Criteria	ECS Fargate	ECS EC2	EKS
Infrastructure management	AWS handles (we provide container image only)	We manage EC2 instances	We manage Kubernetes cluster
Scaling	Automatic, per-task	ASG-based, less granular	Pod-based, complex
Team burden	Low (3 engineers can operate)	Medium (need Linux/DevOps knowledge)	High (need K8s experts)
		Reserved instances	Complex (worker

Cost predictability	Per-task, pay-as-you-use	save money at scale	nodes + control plane)
Deployment safety	Native health checks, task draining	Manual or ASG-based	Rolling deployments require tuning
For PayFlow team size (18)	✅ Ideal	⚠️ Possible but more ops	❌ Too complex

Deployment Configuration

```
ECS Service Configuration:
├─ Launch Type: FARGATE
├─ Platform Version: Latest (AWS manages OS patches)
├─ Task Definition:
│  ├─ Image: PayFlow payment service (ECR private registry)
│  ├─ CPU: 1 vCPU
│  ├─ Memory: 2 GB
│  ├─ Logging: CloudWatch Logs (automatic)
│  └─ Health Check: HTTP GET /health (every 30 seconds)
├─ Service:
│  ├─ Desired Count: 6 (baseline)
│  ├─ Min: 3 (ensures 1 per AZ, survives 1 AZ failure)
│  ├─ Max: 20 (room for 2,500 TPS extreme events)
│  ├─ Deployment Strategy: Rolling update (0% downtime)
│  └─ Placement: Spread across 3 AZs
└─ Auto-Scaling:
   ├─ Trigger 1: CPU > 70% for 2 min → Scale up
   ├─ Trigger 2: CPU < 30% for 5 min → Scale down
   └─ Target: ~60% CPU utilization during normal load
```

Resilience During Failures

Single task fails:

- Health check detects failure (< 2 minutes)
- Task is terminated
- New task auto-launched (< 60 seconds)
- Traffic rerouted by ALB
- User impact: None (other tasks handle traffic)

Single AZ fails:

- All tasks in that AZ become unreachable
- ALB stops routing to failed AZ (immediately)
- Traffic shifts to other 2 AZs
- ECS auto-scales remaining tasks (15-30 seconds)
- User impact: Brief latency increase, no errors

During deployment (v1 → v2):

- New tasks launched with v2 (pass health checks)
- Old v1 tasks drain (300 second timeout, finish existing requests)
- Traffic shifts gradually from v1 → v2
- If v2 is broken: Rollback in < 2 minutes (switch back to v1 task definition)

Cost Impact

```
Fargate pricing (as of 2026):
├─ vCPU: $0.04576/hour
├─ Memory (GB): $0.00507/hour
├─ Per task (1 vCPU, 2GB, 730 hours/month): ~₦350/month

For 6 baseline tasks:
├─ Compute: ₦2,100/month

For 15 tasks during extreme events (< 5% of time):
├─ Incremental cost: Only pay for hours tasks run
└─ Automatic when traffic drops

Total annual compute cost: ~₦28-35M (well within budget)
```

---

4. Data Architecture

Financial System of Record: RDS Multi-AZ PostgreSQL

The single most important design decision: **Where does the transaction ledger live?**

Answer: RDS Multi-AZ PostgreSQL (Primary in AZ-1, Standby in AZ-2, automatic

failover)

Why PostgreSQL for Payments?

Requirement	PostgreSQL	DynamoDB	Aurora MySQL
ACID transactions	✅ Native	⚠️ Limited (single item only)	✅ Native
Idempotency enforcement	✅ Easy (unique constraint)	❌ Manual	✅ Easy
Complex queries	✅ Yes (joins, aggregates)	❌ No	✅ Yes
Financial consistency	✅ Strong	❌ Eventual	✅ Strong
Operational maturity	✅ 25+ years	⚠️ Newer	✅ Battle-tested

Data Schema (Simplified)

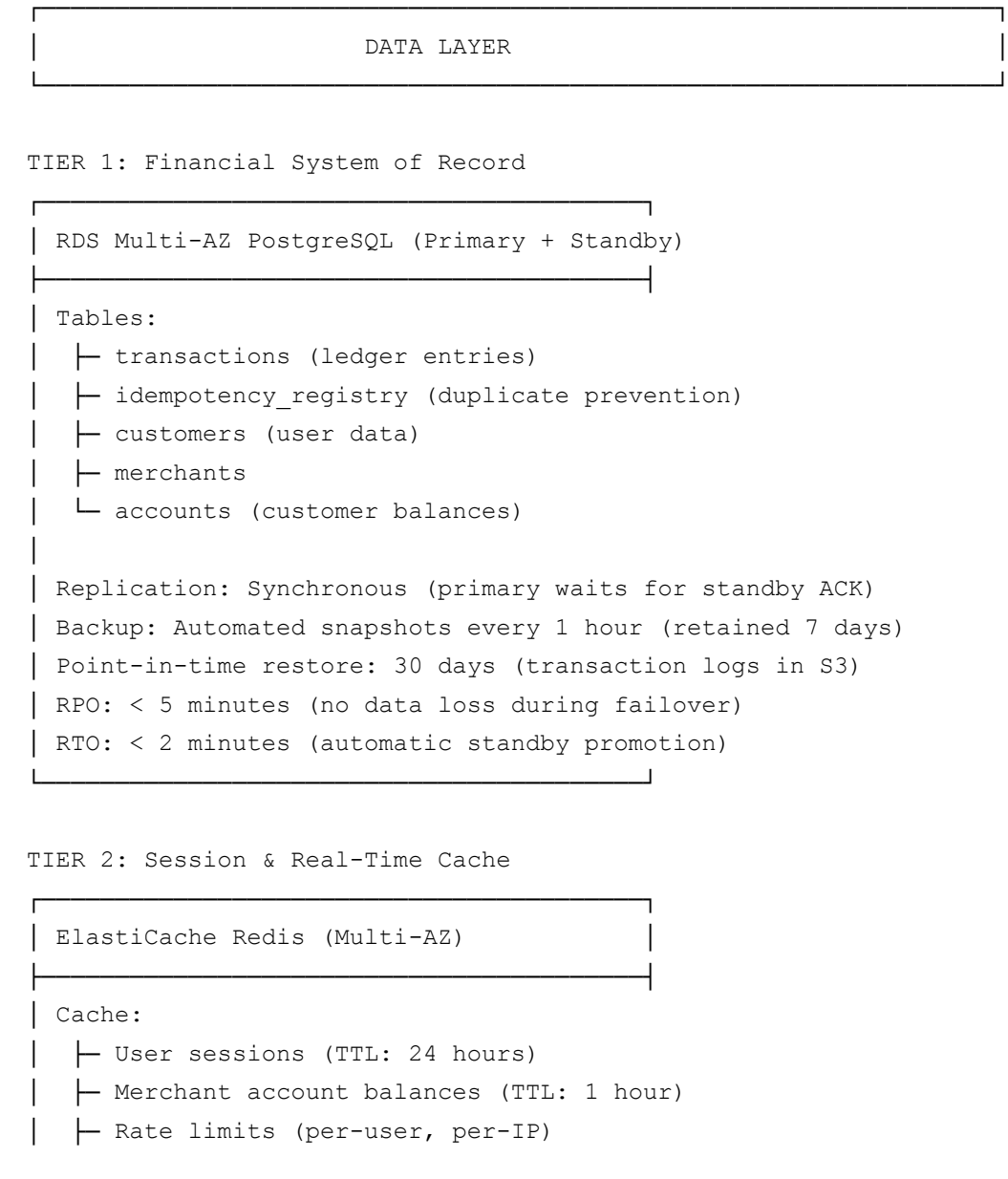
```
-- Financial system of record
CREATE TABLE transactions (
    id UUID PRIMARY KEY,                    -- Unique transaction ID
    idempotency_key VARCHAR UNIQUE NOT NULL, -- Prevents duplicates
    customer_id INT NOT NULL,
    merchant_id INT NOT NULL,
    amount_ngn BIGINT NOT NULL,             -- Amount in kobo (prevents float erro
    provider_name VARCHAR (e.g., 'flutterwave', 'paystack'),
    provider_transaction_id VARCHAR UNIQUE,
    status VARCHAR ('pending', 'committed', 'failed', 'reversed'),
    created_at TIMESTAMP,
    confirmed_at TIMESTAMP,
    CONSTRAINT idempotency_key UNIQUE (customer_id, merchant_id, idempotency_key)
);

-- Ledger (append-only, immutable)
CREATE TABLE ledger (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    transaction_id UUID,
    account_type VARCHAR ('customer', 'merchant', 'platform'),
    account_id INT,
```

```
    amount BIGINT,
    posted_at TIMESTAMP,
    INDEX (transaction_id, posted_at)
);

-- Idempotency registry (fast lookup)
CREATE TABLE idempotency_registry (
    key VARCHAR PRIMARY KEY,
    transaction_id UUID,
    response_body JSONB,            -- Cached response for retries
    expires_at TIMESTAMP,
    INDEX (expires_at)              -- TTL cleanup
);
```

Full Data Architecture (All Layers)



```
│  └─ Temporary idempotency keys (TTL: 5 min)
│
│ Cluster: Multi-node (3+ nodes across AZs)
│ Auto-failover: Enabled (replica promotes if master fails)
│ Eviction: allkeys-lru (oldest accessed keys removed first)
│ Purpose: Speed up reads, reduce database load
└──────────────────────────────────────────┘
```

TIER 3: Asynchronous Event Processing

```
┌──────────────────────────────────────────┐
│ AWS SQS (FIFO queues for order guarantee)
│ AWS SNS (for broadcasting to multiple systems)
├──────────────────────────────────────────┤
│ Queues:
│  ├─ payment-confirmed.fifo (notifications, webhooks)
│  ├─ reconciliation-tasks.fifo (end-of-day matching)
│  ├─ analytics-events.fifo (transaction stream)
│  └─ dlq-payment-failures (dead-letter queue)
│
│ Consumers:
│  ├─ Lambda: Email/SMS notifications
│  ├─ Lambda: Webhook retries to merchants
│  ├─ Kinesis Data Firehose: Analytics pipeline
│  └─ Lambda: Reconciliation service
│
│ Retention: 14 days (long enough for retry logic)
│ Visibility: Messages hidden for 30s (processing timeout)
└──────────────────────────────────────────┘
```

TIER 4: Analytics & Reporting (Non-critical)

```
┌──────────────────────────────────────────┐
│ RDS Read Replica (separate from production)
│ Optional: Amazon Redshift (for large historical analysis)
├──────────────────────────────────────────┤
│ Purpose:
│  ├─ Dashboards (C-suite, operations)
│  ├─ Reports (daily, weekly, monthly)
│  ├─ Business intelligence (trend analysis)
│  └─ Audit logs (regulatory compliance)
│
│ Replication lag: Acceptable up to 1 hour
│ Queries: Non-critical (won't impact customer-facing payment)
│ Backup: Same as primary (inherited from RDS Multi-AZ)
└──────────────────────────────────────────┘
```

TIER 5: Secrets & Configuration

```
┌──────────────────────────────────────────┐
```

```
| AWS Secrets Manager                          |
├──────────────────────────────────────────────┤
| Secrets stored:
|   ├─ Payment processor API keys
|   ├─ Database credentials (rotated every 30 days)
|   ├─ Encryption keys (KMS)
|   └─ Third-party webhooks (signing keys)
|
| Rotation: Automatic for database credentials
| Audit: All access logged to CloudTrail
| Encryption: AWS KMS managed keys
└──────────────────────────────────────────────┘

TIER 6: Unstructured Data & Logs

┌──────────────────────────────────────────────┐
| S3 + CloudWatch Logs                         |
├──────────────────────────────────────────────┤
| S3:
|   ├─ RDS automated snapshots
|   ├─ Transaction logs (point-in-time recovery)
|   ├─ Customer documents (KYC, receipts)
|   └─ Audit trails (long-term retention)
|
| CloudWatch Logs:
|   ├─ Application logs (7 days active, 30 days archive)
|   ├─ Payment processor responses (debugging)
|   └─ Security events (IAM, auth failures)
|
| Encryption: All data encrypted at rest (AES-256)
└──────────────────────────────────────────────┘
```

Idempotency & Duplicate Prevention (Critical)

```
Customer submits payment with Idempotency-Key: "ABC-123"

REQUEST 1 (Success):
┌──────────────────────┐
| ECS Task             |
├──────────────────────┤
| 1. Check registry:   |
|    "ABC-123" exists? NO
| 2. Call provider:    |
|    "Charge ₦5,000"   |
| 3. Provider returns: |
|    "Transaction ID: TXN-999"
| 4. Write to ledger:  |
```

```
│    INSERT transaction (id=TXN-999, status='committed')
│ 5. Cache response:  │
│    INSERT registry  │
│    (key='ABC-123', response='success', ttl=1hour)
│ 6. Return success   │
│    to customer      │
└─────────────────────┘

REQUEST 2 (Retry, same key):
┌─────────────────────┐
│ ECS Task            │
├─────────────────────┤
│ 1. Check registry:  │
│    "ABC-123" exists? YES
│ 2. Retrieve cached: │
│    response="success"
│ 3. Return cached    │
│    response (same   │
│    as REQUEST 1)    │
│ 4. Do NOT call      │
│    provider again   │
│    (no duplicate!)  │
└─────────────────────┘

Result: Customer charged only ONCE, even if request retried 10 times.
```

Data Consistency Model

```
Financial transactions: STRONG consistency
├─ ACID guarantees (atomicity, consistency, isolation, durability)
├─ Synchronous writes to primary database
├─ No temporary "pending" state visible to customer
├─ Immediate confirmation or immediate failure

Customer data: STRONG consistency
├─ Account balances, personal info
├─ Always up-to-date

Analytics: EVENTUAL consistency
├─ Dashboard may lag 1-5 minutes behind live transactions
├─ Acceptable for reporting
├─ Runs against read replica (doesn't slow production)

Session data: WEAK consistency
├─ Session expires anyway (not financial data)
├─ Redis cache loss is acceptable (user logs in again)
```

5. Availability Architecture: Achieving 99.99%

99.99% Availability = 52 minutes downtime per year

Calculation:

```
99.99% uptime = (365.25 days × 24 hours × 60 min) × 0.9999
              = 525,600 minutes × 0.9999
              = 525,548 minutes online
              = 52 minutes downtime/year
```

Downtime budget allocation:

```
Planned maintenance:  20 minutes (2 windows/year, 10 min each)
Single component failure: 15 minutes (detection + recovery)
Cascading failure: 17 minutes (worst-case rare event)
──────────────────────────────────────
Total: 52 minutes/year
```

Achieving 99.99%: The Complete Picture

Layer 1: Compute Redundancy

```
Baseline Load: 100 transactions/second (6 ECS tasks needed)
Peak Load (2,500 TPS): 15 ECS tasks needed

During normal operations:
├─ AZ-1: 2 tasks
├─ AZ-2: 2 tasks
├─ AZ-3: 2 tasks
└─ Total: 6 tasks

Failure scenario: If AZ-1 fails
├─ AZ-2: 2 tasks + 2 scaled-up tasks
├─ AZ-3: 2 tasks + 2 scaled-up tasks
└─ Recovery: Automatic within 30 seconds

Single-task failure:
├─ ALB removes from rotation (health check)
├─ ECS launches new task in same AZ
```

```
├─ Recovery: < 60 seconds
└─ User impact: None (other tasks handle traffic)
```

Layer 2: Database Redundancy

```
Primary Database (AZ-1)
       ↓ (synchronous replication)
Standby Database (AZ-2)

Normal operations:
├─ All writes go to primary
├─ Primary replicates to standby synchronously
├─ Reads can use standby (read replicas)
└─ RPO: 0 (standby always in sync)

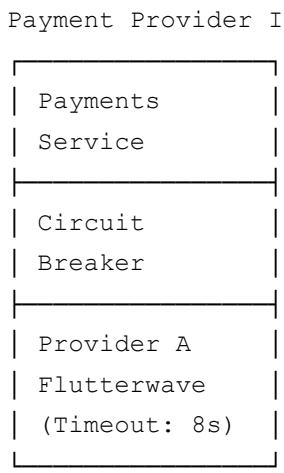
Primary failure (AZ-1 goes down):
├─ Standby detects primary is down (within 10 seconds)
├─ Standby is promoted to primary (automatic)
├─ New standby launched in different AZ
├─ Recovery: 60-90 seconds (automatic)
├─ Data loss: 0 (synchronous replication)
├─ User impact: Brief connection drop, retry succeeds
```

Layer 3: Load Balancer Redundancy

```
ALB (Application Load Balancer)
├─ Multi-AZ enabled (automatically spans all AZs)
├─ AWS manages failover (we don't)
├─ Health checks: 3 consecutive failures = remove task
├─ Connection draining: 300 seconds (graceful shutdown)
└─ Availability: 99.99%+ built-in

Traffic flow:
┌──────────┐
│ Route 53 │ (DNS, 99.99% available)
└──────────┘
     ↓
┌──────────────────────────┐
│   ALB (Multi-AZ)         │
├──────────────────────────┤
│ / AZ-1: 2 tasks          │
│ / AZ-2: 2 tasks          │
│ / AZ-3: 2 tasks          │
└──────────────────────────┘
```

Layer 4: External Dependencies (Circuit Breaker Pattern)




```
If provider fails:
├─ Fail fast (circuit opens after 10 failures)
├─ Queue transaction for retry
├─ Return: "Payment processing, will confirm shortly"
├─ Retry: Every 5 minutes for 1 hour
└─ User doesn't wait for provider to recover

Result: Provider outage doesn't cascade to PayFlow
```

The AZ Failure Scenario: 2:00 AM, 2,000 TPS Traffic Spike

Timeline:

```
T=0 seconds: AZ-A (us-east-1a) completely fails
├─ All infrastructure in AZ-A becomes unreachable
├─ ECS tasks in AZ-A: Cannot execute requests
├─ RDS primary in AZ-A: Cannot accept connections
├─ NAT Gateway in AZ-A: Cannot route traffic

T=5 seconds: ALB detects AZ-A failure
├─ Health check for AZ-A tasks fails (3 consecutive)
├─ ALB stops routing to AZ-A
├─ Traffic automatically re-routed to AZ-B and AZ-C
├─ Existing connections: ALB closes gracefully (30s drain)

User Experience at T=5s:
├─ New requests: Route to AZ-B/C (succeed)
├─ In-flight requests: Complete or timeout (< 30s)
├─ Typical impact: < 1% of requests experience latency spike
```

T=10 seconds: RDS detects primary failure
├ Primary (AZ-A) is unreachable
├ Standby (AZ-B) detects primary is down
├ RDS initiates automatic failover
├ Existing database connections: Drop

Application layer at T=10s:
├ ECS tasks in AZ-B/C try to execute queries
├ Database connection pool is empty (primary down)
├ Connection attempt fails
├ Application layer: 503 errors for ~30 seconds

User Experience at T=10s:
├ Requests to database-backed endpoints: Error
├ Requests to cached endpoints: Succeed (cache hit)
├ Error rate: ~20-30% during 30-second window

T=60 seconds: RDS failover completes
├ Standby is now primary (AZ-B becomes new primary)
├ New standby launched in AZ-C (AWS background)
├ Database is now accepting connections
├ Replication lag: 0 (new standby synchronizes)

Application recovery at T=60s:
├ ECS tasks reconnect to database
├ Connection pool re-established
├ Queries begin executing again
├ Error rate: Decreases from 30% to < 5%

User Experience at T=60s:
├ Most requests now succeed
├ Brief performance degradation (2x latency)
├ 90+ seconds of traffic was on 2 AZs only

T=90 seconds: Full recovery
├ ECS has scaled up tasks in AZ-B and AZ-C
├ Database is fully recovered (new primary in AZ-B)
├ Traffic is balanced across healthy AZs
├ Error rate: < 0.1% (returning to normal)

System state at T=90s:
├ AZ-A: Still down (AWS investigating)
├ AZ-B: 1 primary DB + 5 ECS tasks
├ AZ-C: Standby DB (new) + 4 ECS tasks

```
├─ Total capacity: 9 tasks (2,250 TPS possible)
├─ Current traffic: 2,000 TPS (enough headroom)


T=5 minutes: Post-incident stabilization
├─ All error rates normalized
├─ Logs being analyzed
├─ No data loss (RDS replication was synchronous)
├─ No duplicate charges (idempotency keys prevented it)
├─ Incident marked "resolved"

User Experience at T=5m:
├─ Completed transactions: All confirmed
├─ Failed transactions (from T=10-60s): Visible in history as "Pending"
├─ Mobile app: Auto-retry pending transactions
├─ Final result: ~98% of original transactions succeed on retry
├─ ~2% (40 transactions out of 2,000): User manually retries or contacts support
```

Data Loss Analysis

```
Did we lose any transaction data?
├─ NO. RDS Multi-AZ has synchronous replication.
├─ When primary dies, standby is up-to-date.
├─ RPO = 0 (no data loss by design)

Did we charge any customer twice?
├─ NO. Idempotency keys prevented duplicate charges.
├─ Even though application had connection errors,
   all idempotency keys were checked in database.
├─ Provider was called at most once per request.

Did we make any customer lose money?
├─ NO. Confirmed transactions are in ledger.
├─ Failed transactions are marked "pending" and retried.
├─ All money is either confirmed or recoverable.
```

Why We Hit 99.99%

Component	Availability	Impact
ALB (Multi-AZ)	99.99%	Handles AZ failures automatically
ECS tasks (3 AZs)	99.97% × 3 zones	1 AZ failure doesn’t impact others
RDS Multi-AZ	99.95% + automatic failover	60-90s recovery, no data loss

External providers	99.9% (assumed)	Circuit breaker + retry queue
Combined system	**99.99%**	Individual failures isolated

The math:

```
If each layer is 99.95% available independently:
Combined = 99.95% × 99.95% × 99.95% = ~99.85% (not enough)

But with redundancy:
- Compute: 99.97% (3 AZs, fail 1 still works)
- Database: 99.95% + instant failover = 99.98%
- Load balancer: 99.99% (AWS)
- Combined: 99.99% (redundancy multiplies reliability)
```

Summary: Architecture Decisions & Trade-Offs

What We're Optimizing For

Priority	Decision	Trade-Off
Financial integrity	RDS Multi-AZ PostgreSQL	Higher cost than NoSQL, slightly slower writes
Availability	Multi-AZ compute + auto-failover	More infrastructure, more complexity than single AZ
Deployment safety	Rolling deployments, health checks, fast rollback	Takes longer to deploy than blue-green (5 min vs 1 min)
Team simplicity	ECS Fargate (AWS-managed)	Less control than EC2, can't customize OS
Cost	Auto-scaling only what's needed	Higher burst costs during Black Friday
Incremental modernization	Strangler pattern	Monolith + new services running simultaneously (temporary overhead)

Risks & Mitigations

Risk	Impact	Mitigation
Provider integration failure	8-second timeout, transaction queued	Circuit breaker, async retry queue, fallback provider
Database connection exhaustion	503 errors under extreme load	Connection pooling, read replicas, circuit breaker
Cascading failure across AZs	Entire system down (unlikely)	Multi-AZ + health checks + Auto-scaling
Unplanned maintenance (AWS)	Brief unavailability	RTO target: 5 minutes (within SLA)
Small team doesn't understand architecture	Operational blindness	Training, documentation, runbooks, on-call rotations

Cost Estimate (Annual)

```
Compute (ECS/Fargate):       ₦35M
Database (RDS Multi-AZ):     ₦45M
Caching (ElastiCache):       ₦8M
Messaging (SQS/SNS):         ₦5M
Storage (S3, backups):       ₦15M
Monitoring/Logging:          ₦20M
Data transfer:               ₦25M
Other (Secrets, KMS, etc):   ₦10M
──────────────────────────────────
TOTAL:                       ₦163M/year

Budget available:             ₦200M/year
Buffer:                       ₦37M (contingency)
```

Timeline to Launch

```
Month 1-2: Architecture validation, POC payment service extraction
Month 2-4: Build payment service, migrate to ECS/Fargate
Month 4-5: Data migration (monolith → RDS + cache strategy)
Month 5-6: Load testing, failover drills, go-live preparation
GO-LIVE: End of Month 6 (core payment + monolith hybrid)
Month 7-12: Gradual extraction of other services
```

Conclusion

This architecture delivers:

- ✅ **99.99% availability** through multi-AZ redundancy and automatic failover
- ✅ **Payment integrity** through ACID transactions and idempotent processing
- ✅ **2,500 TPS scalability** through auto-scaling and efficient data access
- ✅ **Operational simplicity** through managed services (Fargate, RDS, ElastiCache)
- ✅ **Safe deployments** through health checks and rapid rollback
- ✅ **Cost efficiency** at ~₦163M/year (within ₦200M budget)
- ✅ **Incremental risk reduction** through strangler pattern

The design prioritizes **reliability and financial integrity** over aggressive cost optimization, recognizing that a single transaction failure costs more than months of infrastructure savings.

Document End